

Sistema recomanador immobiliari mitjançant diàleg

Pràctica 1: millora del sistema basat en regles i del sistema basat en LLM

Natalia Rodríguez Álvarez

Ashlie Ventura Horàn

7 d'octubre de 2026

Índex

1	Introducció	2
1.1	El sistema de partida	2
1.2	Problemes detectats	2
1.3	Orientació de les solucions	2
2	Sistema basat en regles	3
2.1	Casos límit i solucions adoptades	3
2.2	Arquitectura i flux del nou sistema	3
2.2.1	Estructura del codi i modes del diàleg	4
2.2.2	Fase 1: preguntes	4
2.2.3	Fase 2: puntuació i resultats	5
2.2.4	Fase 3: relaxar requisits	6
2.3	Realització i microplanificació	6
2.4	Limitacions	7
3	Sistema basat en LLM	7
3.1	Sistema de partida	7
3.2	Arquitectura general	7
3.3	Flux de la conversa	8
3.3.1	Comprensió de l'entrada, restriccions i relaxacions	8
3.4	Puntuació de cases i relaxació de requisits	9
3.5	Tools de filtratge	9
3.6	Guardrails	10
3.7	Limitacions i possibles millores	10
4	Conclusions	11
4.1	Sistema basat en regles	11
4.2	Sistema basat en LLM	11
4.3	Comparació, limitacions i treball futur	11

1 Introducció

Aquesta pràctica tracta de la construcció d'un assistent conversacional que ajuda un usuari a trobar un habitatge del catàleg proporcionat (25 cases, a `house_data.json`), ja sigui per llogar-lo o per comprar-lo. Parteix d'un xatbot senzill vist a classe i es treballa en dues parts: primer es millora amb un **sistema basat en regles** i, després, es reconstrueix amb un **sistema basat en un LLM** (*Large Language Model*). Les dues parts resolen el mateix problema des de filosofies oposades, i això ens permet comparar què aporta cadascuna.

1.1 El sistema de partida

El punt de partida és un assistent que fa una bateria fixa de preguntes, en l'ordre en què apareixen al JSON, i mostra les cases que coincideixen amb les respostes. La interacció és molt rígida. El sistema espera que l'usuari escrigui *literalment* una de les opcions previstes (o un número) i, si no ho fa, torna a fer la mateixa pregunta sense cap explicació. La cerca és tot o res: una casa només es mostra si compleix *totes* les respostes i, si no n'hi ha cap, el sistema s'excusa i el diàleg s'acaba.

Per a la part basada en LLM, el punt de partida és el xatbot de la sessió de laboratori, que ja delega la interpretació del text al model i li dona una única *tool* de filtratge.

1.2 Problemes detectats

Seguint el mètode iteratiu vist a classe (*exemples, camins, prototipat i proves*), vam escriure diàlegs d'exemple per a perfils diferents (una parella que lloga segons els seus ingressos, un negoci que compra, un usuari indecís) i vam provar entrades límit sobre el prototip. Els problemes que hi vam trobar es poden agrupar en quatre blocs.

Entenen poc i exigeixen massa. El sistema no reconeixia sinònims ni frases amb el mateix significat que una opció («yeah», «I don't want a terrace»), ni números escrits amb lletres («three bedrooms»), ni formats habituals com «100k» o «1,200 €», que no es consideraven iguals a 100000 o a 1200. Això afectava tant la lectura de les respostes com la comparació amb les dades del catàleg, i feia que moltes cerques fracassessin sense motiu. Tampoc tolerava cap errada d'ortografia, per exemple en el nom d'una ciutat.

Validació i cerca massa rígides. D'una banda, s'acceptaven valors fora de rang (nou habitacions, uns ingressos de sis euros); de l'altra, la cerca exigia coincidències exactes fins i tot en camps numèrics amb molt de recorregut, com el preu o els metres quadrats: si cap casa feia exactament 100 m², no n'apareixia cap. Una de les propostes de l'exercici 6 forçava la planta baixa quan l'ús era comercial, cosa poc flexible per a l'usuari i que a més no s'ajustava al catàleg, on hi ha immobles d'ús comercial que no són a planta baixa.

Poc control de la conversa. Si l'usuari s'equivocava, l'única sortida era escriure **quit** (o **q**) i començar de nou, i si ho escrivia sense voler el sistema es tancava sense confirmar. A més, el recorregut era el mateix per a tothom: no hi havia camins de preguntes diferents segons el que anava responent l'usuari, i quan acabava la cerca no es podia veure el detall d'una casa ni tornar a la llista.

Al sistema amb LLM, a més, falta seguretat. El model base no validava prou les respostes, la seva *tool* de filtratge no cobria tots els requisits del catàleg i feia una cerca tot o res, igual que el primer sistema de regles. A més, no hi havia cap control sobre l'entrada (*prompt injection*, llenguatge abusiu, temes fora de l'àmbit) ni sobre la sortida (cases inexistents, fugues del *prompt*).

1.3 Orientació de les solucions

A la **part amb regles** hem reconstruït el sistema perquè entengui l'usuari i el guiï, en lloc d'exigir-li un format. Normalitzem el text i els números, ampliem els sinònims, definim rangs de valors vàlids, adaptem el recorregut de preguntes a les respostes i convertim la cerca en una puntuació que ordena alternatives quan no hi ha cap coincidència exacta. També afegim ordres per corregir i sortir amb confirmació.

A la **part amb LLM** hem fet una divisió de responsabilitats. El model només interpreta el text lliure de l'usuari i, a la fase de resultats, redacta la resposta; tota la resta (quina pregunta toca, validar, puntuar, confirmar) ho fa el codi, de manera que el comportament es pot verificar sense el model. Reaprofitem la lògica de la primera part dins de noves *tools* i hi afegim *guardrails* per capes.

2 Sistema basat en regles

Aquest apartat descriu com hem reconstruït el sistema de partida a partir dels problemes de la introducció. Primer expliquem com hem resolt cada cas límit detectat, i després presentem el nou disseny, amb el seu diagrama de flux, i les millores de realització i de microplanificació.

2.1 Casos límit i solucions adoptades

Comprensió de l'entrada. Per acceptar frases amb un significat similar a les opcions hem creat diccionaris que associen cada etiqueta del catàleg amb els seus sinònims, i que també reconeixen negacions («I don't want a terrace»). Tot el text passa per una única normalització (`normalize_text`). Els números escrits amb lletres es converteixen amb la llibreria `text_to_num`, i s'interpreten els separadors de milers i el sufix `k` ($100k = 100000$), tant a les respostes de l'usuari com a les dades del catàleg. Les expressions d'incertesa («I don't know», «not sure») es detecten abans que la negació, perquè contenen «not» i abans es llegien com un *No*: ara es desa `any` i s'informa l'usuari. A més, hem utilitzat la llibreria `difflib`, que busca similituds entre l'entrada de l'usuari i les paraules i frases dels diccionaris. Això permet acceptar els noms de ciutat o les opcions escrits amb errates, sense exigir una coincidència exacta amb cap sinònim previst, i només s'aplica com a últim recurs, quan cap altra interpretació ha encaixat. Aquesta mateixa idea de flexibilitat és la que apliquem als valors numèrics i a la cerca, com es veu als apartats següents.

Falsos positius lèxics. Alguns d'aquests mecanismes feien interpretacions errònies: «many» contenia «any», «I don't have a budget» contenia «budget» i a «3 kitchens» la *k* es llegia com a mil. Ho hem corregit comparant les paraules soltes per *token* i les frases per *substring*, fixant un ordre explícit de comprovació i acceptant la *k* només com a sufix d'un número.

Validació de valors. Els valors fora de rang o decimals («9 habitacions», «2,5 banys») es rebutgen segons els rangs definits a `NUMERIC_RANGES`, i el missatge d'error explica quin és el rang vàlid perquè l'usuari pugui corregir-ho. Les respostes `any` o «I don't care» ja no s'accepten a totes les preguntes: s'han rebutjat al tipus d'operació, a les preguntes Sí/No del flux i als ingressos, perquè en una pregunta Sí/No equivalien a un *No* (només el *Yes* restringeix) i sense ingressos no es pot calcular la recomanació de lloguer. En canvi, s'accepten a la terrassa, l'ascensor i el preu, on volen dir «no m'importa» o «sense límit».

Preu. El preu es comparava com a text amb el JSON ("250k"), que tenia un format diferent del que escrivia l'usuari, i en lloguer es preguntaven preu i ingressos per separat. Ara el preu és un topall estricte, en comptes d'un valor que hagi de ser exacte. En lloguer, l'usuari pot veure el preu recomanat (el 35 % dels seus ingressos, calculat en el moment) i acceptar-lo o modificar-lo, o bé fixar un límit lliure o deixar el pressupost obert.

Cerca flexible i alternatives. Per evitar que la cerca exigeixi coincidències exactes, el sistema puntua les cases per *desajustos*. Als metres quadrats s'admet un marge asimètric ($-10 / +25 \text{ m}^2$). Quan cap casa coincideix, ja no s'acaba el diàleg: es mostren alternatives ordenades i s'ofereix a l'usuari relaxar alguns requisits.

Ús comercial. Ja no s'imposa la planta baixa. El sistema la recomana i l'usuari ho confirma, i només una resposta afirmativa restringeix la cerca, de manera que un *No* deixa de descartar cases que sí que permetien l'ús comercial. Ho hem fet així perquè la base de dades conté immobles d'ús comercial que no són a planta baixa, i imposar-la els deixava fora injustament.

Control de la conversa. Hem afegit l'ordre `back`, pensada perquè, si el xat s'equivoca o interpreta malament una resposta, l'usuari pugui rectificar sense haver de sortir de la conversa. També es detecta dins d'una frase o amb expressions com «I made a mistake». Hi ha, a més, la sortida (`quit`); totes dues demanen confirmació abans d'actuar, de manera que una errada de teclat ja no tanca la sessió. Quan l'usuari no sap quines ubicacions hi ha, la pregunta anuncia l'ordre `options`, que llista les ubicacions del catàleg i continua esperant la resposta, en lloc d'obligar-lo a endevinar el nom d'un barri o ciutat. Finalment, després de veure les cases, l'usuari pot consultar el detall de diverses i tornar sempre a la llista.

2.2 Arquitectura i flux del nou sistema

El diàleg s'organitza en tres fases consecutives. A la **fase 1** el bot fa les preguntes necessàries per conèixer què busca l'usuari. A la **fase 2** puntua el catàleg i mostra les cases que compleixen exactament tots els requisits; si no

n'hi ha cap, mostra les alternatives més properes, ordenades de millor a pitjor. A la **fase 3** l'usuari pot relaxar els requisits que vulgui per ampliar la cerca. Les fases 2 i 3 estan connectades: des de qualsevol llista de resultats es pot passar a relaxar, i després de relaxar es torna a puntuar (Figura 3).

2.2.1 Estructura del codi i modes del diàleg

El codi té tres capes: una interfície de xat (`ipywidgets`), una classe `DialogEngine` que guarda l'estat de la conversa i un conjunt de funcions sense estat que fan la feina de càlcul (interpretació de respostes, puntuació de cases, relaxació).

L'estat de `DialogEngine` inclou la pregunta actual, les preferències, l'historial que fa servir `back` i, sobretot, el *mode* del diàleg. El mode és necessari perquè l'usuari escriu sempre en el mateix quadre de text, i una mateixa resposta vol dir coses diferents segons el moment: un «2» pot ser el nombre d'habitacions durant les preguntes, però el detall de la segona casa quan s'està mirant una llista. Abans de interpretar cap resposta, el motor mira el mode actiu i la deriva al gestor que toca. Hi ha sis modes, que corresponen a les fases:

- **asking** (fase 1): el bot fa les preguntes del flux i desa les respostes.
- **exact_list** (fase 2): es mostren les cases sense cap desajust. L'usuari pot triar-ne una pel número per veure'n el detall, demanar alternatives o relaxar requisits.
- **alt_list** (fase 2): es mostren les alternatives de tres en tres, quan no hi ha cases exactes o l'usuari en demana més.
- **relax_choice** (fase 3): el bot mostra el menú de requisits relaxables i espera que l'usuari en triï un.
- **relax_more** (fase 3): després de relaxar un requisit, el bot pregunta si en vol relaxar algun altre.
- **finished**: el diàleg ha acabat i el motor ignora noves respostes.

Separar els modes d'aquesta manera fa que cada gestor només hagi d'entendre les respostes que són vàlides en el seu moment, i que afegir una nova etapa només requereixi afegir un mode.

2.2.2 Fase 1: preguntes

El flux de preguntes no és lineal. El camí s'adapta a les respostes de l'usuari (Figura 1) i hi ha dues bifurcacions principals. La primera és l'operació: en compra es pregunta directament el preu màxim, i en lloguer s'obre un bloc de pressupost (Figura 2). La segona és l'ús comercial: si l'usuari el vol, el sistema li recomana planta baixa. El bloc de pressupost en lloguer comença preguntant si l'usuari vol veure el preu màxim recomanat segons els seus ingressos:

- **Si ho vol veure**, el bot en demana els ingressos mensuals, calcula al moment el 35 % i el mostra amb un missatge del tipus *«I recommend that your rent doesn't exceed 35% of your monthly income, which in your case would be 875€/month. Would you like to modify it?»*. Si l'usuari no el vol modificar, aquest import passa a ser el preu màxim i ja no es torna a preguntar cap preu; si el vol modificar, se li pregunta quin preu màxim vol.
- **Si no ho vol veure**, se li pregunta si vol establir un preu límit. Si diu que sí, se li demana el preu (i pot respondre *any* per indicar que no vol límit); si diu que no, el pressupost queda obert (*price = any*).

D'aquesta manera l'usuari decideix si vol guiatge o no, però mai se li fan dues preguntes de preu, i la recomanació és sempre una proposta que pot canviar.

De manera semblant, si l'ús és comercial el sistema recomana planta baixa. Si l'usuari accepta, ja no es pregunta la planta; si la rebutja, es pregunta la planta mínima amb normalitat. La pregunta de la planta mínima és la mateixa per a totes les branques, i es fa just després del bloc d'ús comercial, de manera que els camins convergeixen i el flux no es duplica.

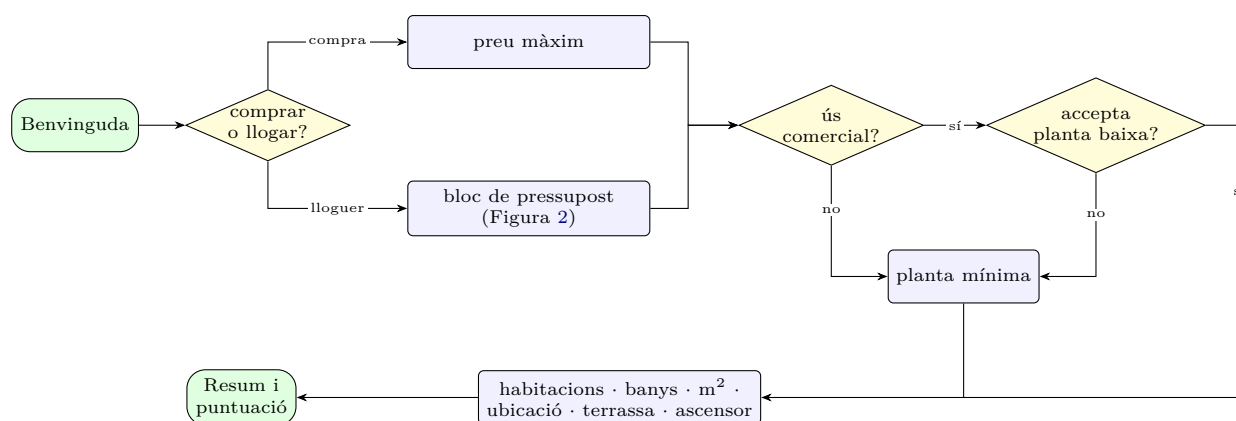


Figura 1: Fase 1 del diàleg: preguntes i camins condicionals. El bloc de pressupost de lloguer es detalla a la Figura 2.

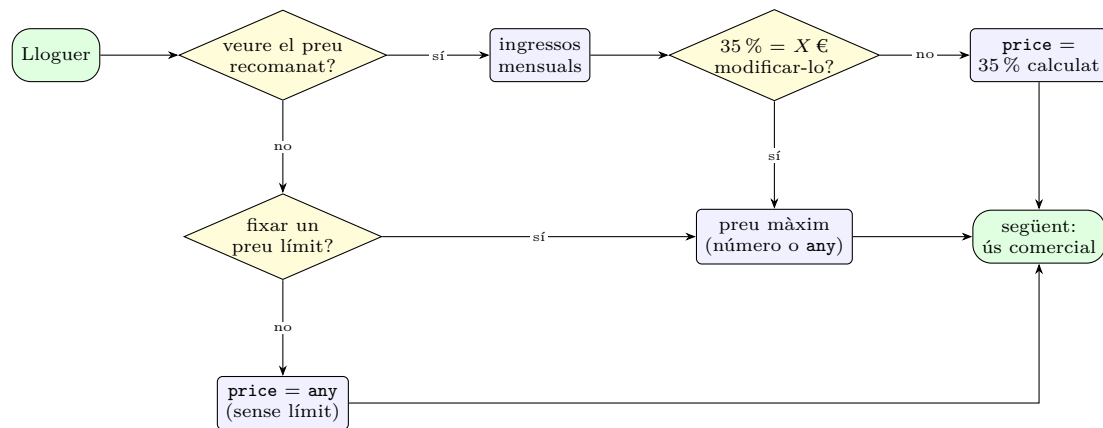


Figura 2: Bloc de pressupost en lloguer. L'usuari tria si vol veure la recomanació (35 % dels ingressos) i si la vol modificar; si no la vol veure, decideix si fixa un preu límit o deixa el pressupost obert.

Resposta any. No totes les preguntes accepten *any* («em dona igual»). En les preguntes *Sí/No* del flux (*see_recommendation*, *modify_price*, *set_limit*, *commercial_use* i *commercial_floor*) *any* equivaldria a *No*, perquè només el *Yes* restringeix la cerca, i per això no té sentit oferir-lo. Tampoc s'accepta a *type*, perquè comprar o llogar és un requisit ferm que decideix quines preguntes es fan després, ni als ingressos, perquè l'usuari ja ha acceptat veure la recomanació i sense ells no es pot calcular. En tots aquests casos el bot demana una resposta clara («*I need a clear answer to this one, 'any' isn't an option here.*») i els missatges d'error ja no suggereixen escriure *any*.

En canvi, *any* s'accepta a terrassa i ascensor (l'usuari pot no tenir preferència), a les preguntes numèriques i d'ubicació i al preu màxim lliure, on vol dir «sense límit». En aquestes preguntes es distingeix entre indiferència i negació: «*I don't mind*» o «*whatever*» es desen com a *any*, però «*I don't want any terrace*» és un *No*, perquè la paraula «*any*» dins d'una frase amb negació no expressa indiferència.

Ordre options. A la pregunta d'ubicació s'hi afegeix l'avís «*If you'd like to see the available options, type 'options'.*». Quan l'usuari escriu *options* (o una frase com «*show me the options*»), el bot llista les ubicacions del catàleg i torna a esperar la resposta, sense avançar de pregunta ni modificar les preferències. Així l'usuari no ha d'endevinar el nom d'un barri o d'una ciutat.

2.2.3 Fase 2: puntuació i resultats

El sistema de partida només mostrava les cases que complien tots els requisits de forma exacta: el preu i els metres quadrats havien de coincidir amb el valor demanat. Això no és realista, perquè és molt probable que cap casa coincideixi exactament i que l'usuari es quedi sense res. Per això hem fet dos canvis. Primer, hem modificat algunes preguntes: el preu passa a ser un *preu màxim* i els metres quadrats accepten un marge. Segon, hem introduït un sistema de puntuació, *score_houses*.

Aquesta funció recorre el catàleg una sola vegada i, per a cada casa, calcula la llista dels seus *desajustos*, és a dir, els requisits de l'usuari que la casa no compleix. Cada desajust té una gravetat entre 0 i 1, definida a la Taula 1. Com que els criteris només s'escriuen un cop, la mateixa llista serveix per mostrar resultats, ordenar alternatives i calcular la relaxació.

Abans de puntuar, el sistema aplica els *requisits durs* (*passes_hard_requirements*), que no es negocien mai ni es poden relaxar: el tipus d'operació i, si l'usuari vol ús comercial, que la casa el permeti. Les cases que no els compleixen queden fora de la llista. La planta baixa és un cas a part: si l'usuari l'ha acceptat per a l'ús comercial, no filtra el catàleg abans de puntuar, sinó que es tracta com un desajust de gravetat 1 que *bloqueja* la casa. Una casa bloquejada no surt ni a les alternatives, però, a diferència dels requisits durs, aquest requisit sí que es pot relaxar a la fase 3.

Amb els desajustos calculats, els resultats es presenten així:

- Una casa és *exacta* si no té cap desajust. Es mostren totes les exactes, i l'usuari pot triar-ne una pel número (per veure'n el detall complet) o demanar alternatives.
- Si no n'hi ha cap, el sistema ho diu i passa directament a les tres millors alternatives. Amb *more* es poden veure les tres següents.
- Les alternatives s'ordenen pel nombre de desajustos, després per la gravetat total i, per desempatar, per l'identificador de la casa.

Requisit	Hi ha desajust si...	Gravetat
Pressupost	El preu supera el topall (preu directe o 35 % dels ingressos)	$\min\left(1, \frac{\text{preu} - \text{topall}}{\text{topall}}\right)$
Metres quadrats	La casa surt del marge: 10 m ² menys o 25 m ² més del demanat	excés sobre el marge / 25, màx. 1
Planta mínima	La casa és per sota de la planta demanada	$\min\left(1, \frac{\text{demanada} - \text{real}}{\max(\text{demanada}, 1)}\right)$
Habitacions, banys	El nombre no coincideix	$\min(1, \text{diferència} /2)$
Ubicació	La zona és diferent	0,6 fix
Terrassa, ascensor	L'usuari els vol i la casa no els té	0,3 fix
Planta baixa (ús comercial)	La casa no és de planta 0	1 (bloqueja; no apareix a les alternatives)

Taula 1: Desajustos i gravetat.

2.2.4 Fase 3: relaxar requisits

L'usuari pot demanar relaxar requisits (**relax**) des de qualsevol llista de la fase 2, i en tria *un cada vegada*, pel número o per paraula clau («the budget», «terrace»). El menú només mostra els requisits actius i, al costat de cadascun, quantes cases més apareixerien, que són les que només fallen en aquest camp. Aquest recompte es calcula a partir de la mateixa llista de desajustos, sense tornar a cercar.

Relaxar el pressupost puja el topall un 10 % (**BUDGET_RELAXATION_PERCENT**); la resta de requisits passen a **any**. Entre els requisits relaxables hi ha la planta baixa de l'ús comercial: tot i que bloqueja les cases que no hi són, l'usuari pot decidir renunciar-hi i veure-les. En canvi, el tipus d'operació i l'ús comercial *Yes* no es poden relaxar, perquè són requisits durs. Després de cada canvi, el bot pregunta si es vol relaxar res més i, quan l'usuari diu que no, es torna a puntuar amb les preferències noves. Els canvis són acumulatius.

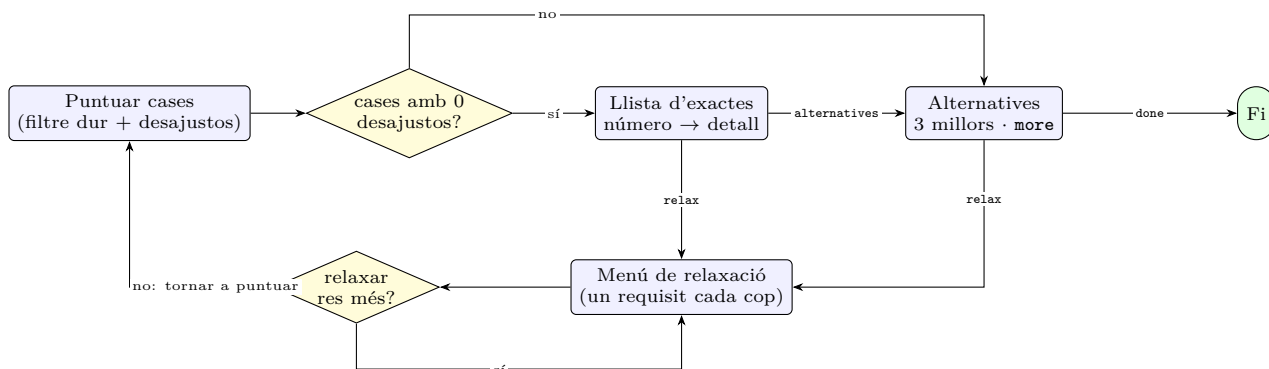


Figura 3: Fases 2 i 3: puntuació, presentació de resultats i relaxació. Des de qualsevol llista, **done** acaba el diàleg i q demana sortir.

2.3 Realització i microplanificació

Hem treballat la generació del text amb les etapes clàssiques de NLG. Els problemes que hi vam trobar són que el bot repetia sempre les mateixes frases, no mostrava que havia entès l'usuari i abocava massa informació de cop.

Estructuració del contingut. Les llistes mostren una sola línia per casa (*House 3 · Santa Coloma de Gramenet · 800€/month*) i el detall complet només surt quan l'usuari tria una casa. A les alternatives s'hi afegeix *differs in*, amb els desajustos explicats en text, perquè l'usuari vegi per què no és una coincidència exacta.

Lexicalització. Cada pregunta té tres formulacions equivalents i se'n tria una a l'atzar, de manera que el diàleg no sona repetitiu. La pregunta amb la recomanació de preu és una plantilla amb un marcador que se substitueix en el moment de preguntar, i així el missatge mostra sempre el 35 % dels ingressos que l'usuari acaba de donar. Les confirmacions curtes (*Got it — 3 bedrooms.*) usen quatre plantilles i afegeixen la unitat segons el camp.

Agregació. Abans de cercar, el sistema resumeix en una sola frase totes les preferències (*Renting, 3 bedrooms, Barcelona, up to 1200€ — searching...*) i només hi inclou allò que l'usuari ha fixat.

Expressions referencials i guiatge. L'usuari pot triar una casa o un requisit amb un número, un ordinal («the second one») o una paraula clau. A més, cada llista indica quines ordres són vàlides *en aquell moment* (more només apareix si queden alternatives) i els errors expliquen el rang acceptat en lloc de repetir la pregunta.

2.4 Limitacions

El sistema segueix sent basat en regles: només entén el vocabulari que hem previst (anglès, amb un lèxic finit), de manera que una resposta en català o en castellà es tracta com una opció no reconeguda. Fins i tot dins d'aquest vocabulari poden haver-hi errors en associar la resposta de l'usuari amb els diccionaris, ja que és impossible preveure totes les maneres de dir una mateixa cosa. A més, els pesos de gravetat i el 35 % dels ingressos són regles heurístiques que no hem validat amb usuaris reals. Aquestes limitacions són el punt de partida natural per a la segona part, on un LLM interpreta el text lliure sense dependre d'un lèxic tancat.

3 Sistema basat en LLM

El segon sistema substitueix la lògica de diàleg fixa per un model de llenguatge (*Large Language Model*, LLM), amb el qual aconseguim interpretar el text lliure de l'usuari i cridar *tools* per consultar el catàleg. A més, hem afegit uns *guardrails* per aconseguir més seguretat en el model.

Hem treballat amb un model petit sobre el mateix catàleg de 25 habitatges de la primera part. Les respostes són en anglès, igual que al sistema de regles, però ara també hem aconseguit que es pugui escriure en català o en castellà.

3.1 Sistema de partida

El xatbot de partida és el de la sessió de laboratori, el qual al contari que el de regles, ja tenia interpretació de text lliure. Els problemes que hi vam detectar (validació insuficient, una única tool de filtratge amb cerca de tot o res, i cap control de seguretat a l'entrada ni a la sortida) són els que intentem resoldre en aquest sistema.

3.2 Arquitectura general

La decisió de disseny ha estat una de les més rellevants per aconseguir que el model no s'inventi respostes. Per això el LLM només interpreta i, a la fase de resultats, redacta. Totes les altres tasques (quina pregunta toca, validació de valors, puntuació de cases, confirmacions, benvinguda i tancament) les fa el codi. Així el sistema és més fiable i el seu comportament es pot verificar sense el model.

La Figura 4 mostra què passa a cada torn de l'usuari.

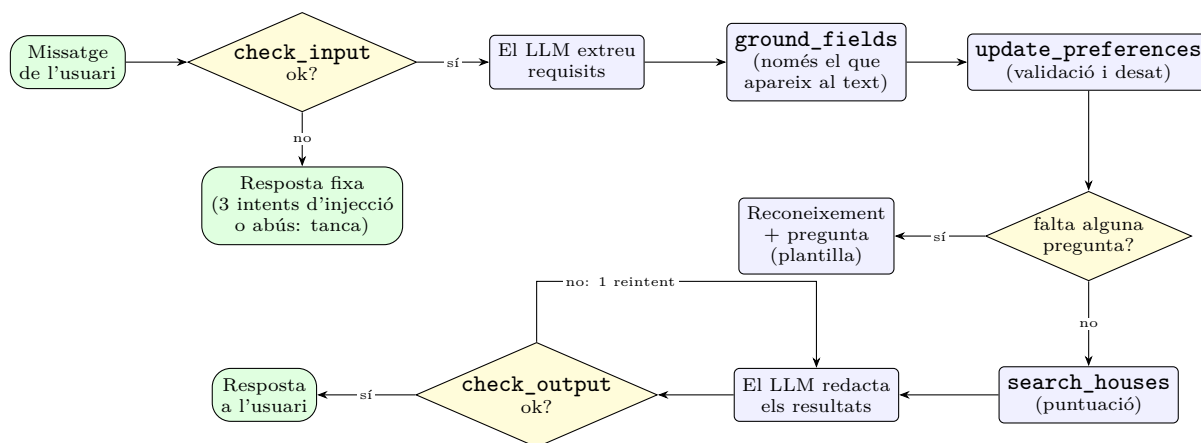


Figura 4: Flux d'un torn de la fase de preguntes. El codi controla totes les caixes excepte les dues del LLM: l'extracció de requisits i la redacció dels resultats.

A la fase de resultats el torn és més curt i el missatge passa pel `check_input` i va directe al LLM, que decideix quina tool cridar, i la resposta passa pel guardrail `check_output`.

3.3 Flux de la conversa

El flux de conversa del nostre sistema organitza el recorregut del xatbot, des de la benvinguda fins al tancament. Està controlat pel codi, que decideix en cada moment quina pregunta toca, valida les respostes de l'usuari i determina quan ja es disposa de tota la informació per passar a la fase de resultats. Això evita que el model hagi d'improvisar l'estructura del diàleg i s'inventi coses, i alhora garanteix que no es quedi cap dada necessària per recollir.

Dins d'aquest recorregut, el LLM només s'encarrega d'interpretar el text lliure de l'usuari i, a la fase final, de redactar els resultats. A continuació descrivim les fases principals del flux i com interactuen amb el model:

- **Inici.** El bot dona la benvinguda amb el `start_message` del JSON, explica que es pot descriure la casa lliurement en un sol missatge i recorda les ordres `back` i `quit`.
- **Informació lliure.** L'usuari pot donar diversos requisits alhora. El LLM els extreu i el bot només pregunta el que encara falta, en l'ordre que fixa `next_missing_question`. Hem optat per aquest ordre per mantenir la coherència entre les preguntes. Comença amb la pregunta que condiciona la cerca (si es vol llogar o comprar), seguida de les característiques més bàsiques, com el nombre de lavabos o d'habitacions. Les característiques addicionals de la casa (ascensor, terrassa, etc.) es deixen per al final.
- **Reconeixement.** Abans de cada pregunta el bot confirma el que ha entès (*Got it: renting, in Barcelona and 2 bedrooms*) amb plantilles que van rotant, en lloc de deixar que el LLM les redacti. Ho hem fet així perquè les respostes siguin més humanes ja que abans el bot no mostrava que havia entès el que s'havia dit anteriorment, i en una conversa entre persones agrada rebre una mostra de comprensió de l'altre. A més, quan era el LLM qui escrivia aquestes confirmacions, de vegades s'inventava coses que l'usuari no havia dit.
- **Bifurcacions.** Si l'usuari vol llogar, es pregunten els ingressos i el lloguer màxim; si aquest supera el 33 % dels ingressos (`RENT_INCOME_RATIO`), el bot ho avisa i recomana baixar-lo, però l'usuari pot mantenir-lo. En canvi, l'ús comercial només es pregunta si l'usuari vol comprar i, si respon que sí, també es pregunta si accepta una planta baixa. En aquest cas ja no cal preguntar la planta mínima.
- **Recomanació de cases.** Quan s'acaba el procés de preguntes, el bot mostra per pantalla el conjunt de cases que s'ajusten a les preferències de l'usuari. Igual que passa amb el model basat en regles, no sempre hi ha una casa que quadri exactament, i per això fem servir un sistema de puntuació (explicat amb més detall a l'apartat 3.4). A més, quan es mostren cases que només compleixen part de les preferències, el bot ofereix la possibilitat de relaxar-ne algunes per obrir la porta a trobar una casa amb totes les característiques que l'usuari vol.
- **Correcció i sortida.** `back` restaura les preferències i l'historial; `quit` demana confirmació. Si l'usuari diu que no vol res més, el bot mostra un resum de la cerca i el `end_message`. Cada `start_chat_session()` crea una `Session` nova, de manera que cap conversa hereta l'estat de l'anterior.

3.3.1 Comprensió de l'entrada, restriccions i relaxacions

Una de les parts que més hem treballat és l'enteniment de les entrades de l'usuari. Els problemes principals que hem detectat i corregit es poden agrupar en quatre blocs:

- **Format dels números.** S'accepten números escrits de moltes maneres: sols (3) o acompanyats de les seves unitats (3 euros); en xifres amb diferents formats (2000, 2.000, 2,000 o 2k); i fins i tot escrits amb lletres. Un conjunt de funcions s'encarrega d'interpretar-los i de deixar-los en format numèric (enter o decimal).
- **Rangs de valors vàlids.** Un dels problemes més notables del sistema de partida era que s'acceptava qualsevol número a qualsevol pregunta. Per evitar-ho, hem definit rangs de valors que considerem vàlids i realistes per a cada camp. Aquests rangs es poden modificar amb molta facilitat, ja que estan recollits en diccionaris. És a dir, hem restringit les respostes numèriques a valors raonables, i quan n'arriba un de fora de rang, el bot ho avisa en lloc d'acceptar-lo. El rang del preu depèn de si es vol llogar o comprar.
- **Flexibilitat amb el text.** Amb el text hem fet el contrari i hem relaxat la gestió de les faltes d'ortografia, de manera que les paraules mal escrites per error s'accepten igualment. També hem creat un diccionari d'àlies de les ciutats i barris on vol viure l'usuari, per donar-li més flexibilitat a l'hora d'escriure-ho de moltes maneres diferents.
- **Opcions múltiples.** Per últim, hem permès donar diverses opcions. Si l'usuari dubta entre dos valors, el xatbot no n'escull un, sinó que els accepta tots, com a *2 or 3 bedrooms*, *2-3 or three or four bathrooms*.

La Taula 2 resumeix la comprensió de l'entrada, que és on el sistema de partida fallava més.

Entrada	Abans	Ara	Funció
«3 euros», «60 m2»	No s'entenia la unitat	3, 60	<code>parse_number</code>
«2000», «2.000», «2,000», «2k»	«2.000» es llegia com 2 i «2,000» no s'entenia	2000 en els quatre casos	<code>parse_number</code>
«six hundred», «twenty-five»	Es llegia com 6 / 25 incomplet	600, 25	<code>normalize_number_words</code>
«2 or 3 bedrooms», «2-3», «between 2 and 4», «three or four bathrooms»	Es guardava com 2	Llista d'opcions ([2, 3], [2, 3], [2, 3, 4], [3, 4]); la cerca n'accepta qualsevol	<code>parse_options</code> , <code>apply_number_options</code>
«2 and terrace» (es pregunta pels lavabos)	El 2 es guardava com a habitacions	Un número només és d'un camp si hi ha la seva paraula; si no, respon la pregunta pendent	<code>ground_fields</code> , <code>fill_pending_from_bare_number</code>
«no and no elevator» (es pregunta per la terrassa)	Es perdia el <i>no</i> de la terrassa	Terrassa: No; ascensor: No. Paraula + negació, també amb errates	<code>fill_pending_yes_no</code> , <code>yes_no_mentions</code>
«6 euros» d'ingressos, «10 bedrooms»	S'acceptava	«6 € doesn't look like a valid monthly income: it should be between 500 € and 100,000 €.»	<code>validate_number</code> (rangs a <code>NUMERIC_RANGES</code> i <code>PRICE_RANGES</code>)
«betrooms», «elavtor»	—	Es corregeix a «bedrooms», «elevator»	<code>fix_typos</code>
«bcn», «barna», «hospitalet», «barcelna»	No s'entenia	Barcelona, Barcelona, L'Hospitalet de Llobregat, Barcelona; si no hi ha coincidència, es llisten les zones disponibles	<code>resolve_location</code> (<code>LOCATION_ALIASES</code>), <code>resolve_locations</code> , <code>fuzzy_match</code>

Taula 2: Millores en la interpretació de l'entrada.

3.4 Puntuació de cases i relaxació de requisits

Les tools reutilitzen sense canvis el motor de la primera part. La funció `score_houses` calcula per a cada casa la llista de desajustos amb les gravetats de la Taula 1, i separa les cases exactes de les alternatives, que s'ordenen igual que a la fase 2 del sistema de regles. La relaxació també és la mateixa (fase 3, Figura 3): un requisit cada vegada, amb el pressupost un 10 % més alt i la resta a *any*, i amb el recompte de cases que s'afegirien. Allò que canvia és qui ho crida: `search_houses` i `relax_requirement` llegeixen les preferències de la sessió, de manera que el LLM no pot modificar-ne els criteris i aconseguim encara més robustesa en el model.

3.5 Tools de filtratge

Hem substituït la tool original (`filter_real_estate`, amb només cinc filtres estrictes) per quatre tools que cobreixen tots els requisits del catàleg. Les descrivim al model amb el format JSON de *function calling* (nom, descripció i paràmetres amb tipus i valors admesos):

Tool	Funció
<code>update_preferences</code>	Valida i desa els requisits que l'usuari acaba de dir (tipus, preu, ingressos, ús comercial, planta, habitacions, banys, m ² , ubicació, terrassa, ascensor). Accepta <i>any</i> , opcions (2 or 3) i àlies de zones (<i>bcn</i>). Retorna els camps desats, els errors i la propera pregunta
<code>search_houses</code>	Puntua tot el catàleg amb la lògica de l'apartat 3.4: les cases amb 0 desajustos són exactes i la resta, alternatives ordenades; admet paginar
<code>get_house_details</code>	Retorna el detall complet d'una casa a partir del seu identificador
<code>relax_requirement</code>	Sense camp, llista els requisits que es poden relaxar i quantes cases més apareixerien; amb camp, relaxa aquell requisit

Taula 3: Tools del sistema.

Qui crida les tools depèn de la fase de la conversa. A la fase de preguntes no ho decideix el LLM, aquí el codi crida `update_preferences` amb el que ha extret i, quan ja no falta cap dada, llança `search_houses` automàticament. A la fase de resultats sí que és el LLM qui decideix quina tool cridar, segons el que demani l'usuari es pot donar el detall d'una casa, més alternatives, relaxar un requisit o canviar-ne algun.

Les tools reutilitzen la lògica del sistema basat en regles i les funcions de comprensió de l'entrada que hem comentat abans:

- `update_preferences` és on s'apliquen la validació de rangs, els àlies i les opcions múltiples. Si un valor no és vàlid no es desa, i l'error explica el motiu a l'usuari. També manté la coherència de les dades i si l'usuari canvia

de lloguer a compra, el sou deixa de ser necessari; si canvia de compra a lloguer, es descarta l'ús comercial; i si el preu que ja tenia desat no és vàlid per al nou tipus d'operació, s'esborra i es torna a preguntar.

- **search_houses** no rep filtres com a arguments, sinó que llegeix les preferències de la sessió. Així el model no pot oblidar un requisit ni inventar-ne un en cercar. Si falta alguna dada, es nega a cercar i diu quina falta. Retorna les cases exactes i una pàgina de 3 alternatives, amb el que no compleix cadascuna.
- **get_house_details** i **relax_requirement** són les dues tools noves. La segona aplica la relaxació descrita a l'apartat 3.4, i després de relaxar, el model torna a cridar **search_houses**.

El *system prompt* no és fix i es reconstrueix a cada iteració amb `build_system_prompt()`, que hi inclou la fase (recollida de requisits o resultats), les preferències desades i unes regles curtes, com ara només esmentar cases que surtin a les tools. El model genera de manera determinista (`do_sample=False`). El bucle de crides a tools (`query_model_with_tools`) té un màxim de 5 iteracions per torn, bloqueja una crida repetida amb els mateixos arguments i, si el JSON de la tool està mal format, ho explica al model perquè es corregeixi. Si la tool no existeix o els arguments són incorrectes, `call_tool` no falla, retorna un error en JSON perquè el model es corregeixi.

3.6 Guardrails

Hem afegit mesures de seguretat en diverses capes (Taula 4), perquè detectar atacs amb patrons no és infal·libre i cap capa és suficient per si sola.

Capa	On	Què fa
Entrada	<code>check_input</code>	S'executa abans que el missatge arribi al model. Neteja els caràcters de control i bloqueja els missatges de més de 300 caràcters, la injecció de prompt (també amb tokens especials, etiquetes de rol falses o blocs de codi), el llenguatge abusiu i els temes fora de l'àmbit. Després de tres intents d'injecció o abús, tanca la conversa
Context	<code>build_system_prompt</code>	El text de l'usuari només entra com a missatge user . Les regles es reconstrueixen a cada iteració i indiquen al model que ignori qualsevol ordre de l'usuari que intenti canviar-les
Extracció	<code>ground_fields</code>	Només accepta un valor extret pel LLM si l'usuari l'ha dit realment. Un número ha d'aparèixer al text i anar acompanyat de la seva paraula (tret que respongui la pregunta pendent), el tipus d'operació requereix <i>buy</i> o <i>rent</i> , i un <i>any</i> requereix que l'usuari mostri indiferència. Si el LLM no n'extreu res, <code>rule_based_extract</code> interpreta les respostes curtes
Tools	<code>update_preferences</code> , <code>call_tool</code> , <code>query_model_with_tools</code>	Validen el tipus i el rang dels valors. Si una tool no existeix o els arguments són incorrectes, retornen un error en JSON perquè el model es corregeixi. A més, el bucle es limita a 5 iteracions i es bloquegen les crides repetides
Sortida	<code>check_output</code> , <code>guarded_response</code>	S'executa després del model. Rebutja les respostes buides o de més de 1 500 caràcters, les que mostren el prompt o la sintaxi de les tools i les que citen cases que no han aparegut als resultats. El model té un reintent i, si torna a fallar, es mostra un text de reserva generat pel codi

Taula 4: Guardrails per capes.

Les capes són independents, i això és el que les fa útils. Si una injecció s'escapa del filtre d'entrada, encara ha de passar l'extracció, que només accepta el que l'usuari ha dit, i la validació de les tools. Si el model acaba redactant una resposta incorrecta, per exemple citant una casa que no ha sortit a cap resultat, el filtre de sortida la rebutja; després d'un reintent, el sistema respon amb un text generat pel codi (`render_results`). El filtre d'entrada és el més fàcil de saltar perquè es basa en patrons, però el que realment protegeix el sistema és que el LLM no decideixi res crític, no escull la pregunta, no valida els valors i només consulta tools que llegeixen les preferències de la sessió.

3.7 Limitacions i possibles millores

Tot i les millores respecte al sistema de partida, el xatbot té limitacions que hem identificat:

- **Model petit.** Per això hem reduït tant el que li demanem, i tot i així de vegades no extreu requisits que l'usuari ha dit, o ho fa amb un format inesperat. Les regles deterministes ho compensen en part, però no del tot.
- **Patrons fixos.** Els filtres d'entrada i de sortida es basen en expressions regulars. Poden deixar passar atacs formulats d'una manera que no hem previst i, al revés, bloquejar missatges legítims que contenen una paraula sospitosa.

- **Correcció només a les preguntes.** L'ordre `back` només funciona a la fase de preguntes. A la de resultats, l'usuari ha de demanar el canvi en text lliure o relaxar un requisit.
- **Idioma.** El xatbot només parla un sol idioma, l'anglès.

Com a possibles millores, es podrien ajustar els pesos de la puntuació amb l'opinió d'usuaris reals, ampliar els idiomes admesos i millorar la seguretat davant unjeccions per obtenir un model més robust davant d'atacs reformulats.

4 Conclusions

En aquesta pràctica hem partit d'un xatbot immobiliari molt rígid i l'hem reconstruït de dues maneres diferents. Els dos sistemes resolen el mateix problema (ajudar un usuari a trobar un habitatge del catàleg) amb filosofies oposades, i això ens ha permès veure què aporta cadascuna i on falla.

4.1 Sistema basat en regles

Hem millorat el sistema de partida de manera clara. Ara entén sinònims, números en formats diversos i errates, valida els valors, adapta les preguntes al camí de l'usuari i, quan no hi ha cap coincidència exacta, mostra alternatives ordenades i permet relaxar requisits. El canvi que més ha pesat ha estat passar d'una cerca de tot o res a una *puntuació per desajustos*, que ens ha servit per ordenar alternatives, explicar-les i calcular què es guanya en relaxar un requisit.

Tot i això, el sistema segueix sent rígid, i és la conclusió principal que en treiem. Els diccionaris permeten reconèixer moltes maneres de respondre, però són finits: només entenen les formes que hem previst, i qualsevol altra queda fora. Cada vegada que hem trobat un cas que fallava, la solució ha estat afegir una nova regla, un nou sinònim o un nou ordre de comprovació, i això no té fi, perquè sempre hi ha una manera de dir les coses que no hem anticipat. A més, els mecanismes es poden interferir entre ells, com es veu amb els falsos positius lèxics («many» que contenia «any», o la *k* de «kitchens»).

El tractament de les faltes d'ortografia mostra bé aquest problema. La comparació aproximada de cadenes evita molts errors, però és sensible: depèn d'un llindar de similitud, i si és massa estricta rebutja respostes correctes i si és massa tolerant n'accepta d'incorrectes. Per això, tot i haver-nos esforçat a no donar errors davant d'una errata, el sistema encara pot fallar o no entendre una resposta perfectament raonable, i l'usuari ho percep com una conversa poc natural.

4.2 Sistema basat en LLM

El sistema amb LLM resol justament aquesta rigidesa: l'usuari pot descriure el que busca en text lliure, en un sol missatge i sense seguir un guió, i el model interpreta formes de parlar que cap diccionari havia previst. El preu d'aquesta flexibilitat és que el model pot inventar-se informació, extreure valors que l'usuari no ha dit o ser manipulat amb *prompt injection*.

Per això el que ha funcionat millor no és deixar més feina al model, sinó *limitar-la*. El LLM només interpreta i, a la fase de resultats, redacta; les decisions que han de ser deterministes (validar rangs, puntuar cases, mantenir la coherència de les preferències) les fa el codi. Reaprofitar la lògica de la primera part dins de les *tools* i afegir *guardrails* per capes ens ha permès combinar la comprensió del llenguatge del LLM amb la fiabilitat del sistema de regles.

4.3 Comparació, limitacions i treball futur

Els dos enfocaments són complementaris. El de regles és previsible i es pot verificar sense cap model, però és fràgil davant del llenguatge natural; el del LLM és flexible, però requereix mesures de control perquè sigui fiable. En un sistema real, la millor opció sembla combinar-los, com hem fet a la segona part: el model per entendre l'usuari i el codi per garantir el comportament.

Les dues parts tenen limitacions. Els pesos de gravetat dels desajustos i el 35 % dels ingressos recomanat per al lloguer són regles heurístiques que no hem validat amb usuaris reals, i el catàleg és petit (25 cases). A la part amb LLM, la detecció d'atacs es basa en patrons i, per tant, no és infal·lible. Com a treball futur, caldria avaluar tots dos sistemes amb usuaris reals, per ajustar els pesos i mesurar quant sovint cada sistema no entén una resposta, i ampliar el suport a altres idiomes.